

Programmation Orientée Objet en Python

Sans ordinateur · Sans document · Sans calculatrice

NOM _____
 PRÉNOM _____
 CLASSE _____
 DATE _____

🕒 **Durée : 1h** ✎ Écrire proprement, une instruction par ligne 🐍 Python uniquement — pas de pseudo-code
 ✂ **and / or / not** en minuscules ❌ Pas de `new`, pas de `AND`, pas de `&&`

★ Automatismes syntaxiques

★★ Logique et comparaisons

★★★ Encapsulation et méthodes

★★★★ Listes d'objets

★★★★★ Niveau Baccalauréat

PARTIE 01 Automatismes syntaxiques ★

8 points

Exo 1.1 Le rituel du constructeur

★ 2 pts

On veut modéliser une **Voiture** avec deux attributs : `marque` (chaîne de caractères) et `vitesse_max` (entier).

Écris uniquement le constructeur de cette classe. Rien d'autre.

⚠ **Rappels** : Le constructeur s'appelle `__init__` (deux underscores de chaque côté) · Son premier paramètre est toujours `self` · Chaque attribut se déclare avec `self.nom = valeur` · Pas de `return`

TA RÉPONSE

2 LIGNES MINIMUM

1
 2
 3
 4

Exo 1.2 Instanciation correcte

★ 2 pts

On dispose de la classe suivante :

```
class Livre:
    def __init__(self, titre, pages):
        self.titre = titre
        self.pages = pages
```

Écris la ligne qui crée un objet `mon_livre` représentant "Dune" avec 412 pages.

⚠ **Rappel** : On instancie avec `NomClasse(arg1, arg2)` — on n'écrit jamais `__init__` ni `self` lors de l'appel.

TA RÉPONSE

1 LIGNE

1

Exo 1.3 Chasse au bug — trouve les 3 erreurs

★ 3 pts

Le code ci-dessous contient **3 erreurs de syntaxe POO**. Entoure chaque erreur et écris la version corrigée dans le cadre.

```
class Compte:
    def __init__(self, titulaire, solde):
        titulaire = titulaire
        self.solde = solde

    def deposter(montant):
        self.solde = self.solde + montant

    def afficher(self):
        print(self.solde AND self.titulaire)
```

⚠ **Rappel** : Toute méthode prend `self` en premier paramètre · Les attributs s'écrivent `self.nom` · Les opérateurs logiques Python sont en minuscules : `and`, `or`, `not`

VERSION CORRIGÉE COMPLÈTE

RÉÉCRIRE LA CLASSE ENTIÈRE

1
2
3
4
5
6
7
8
9
10

Exo 1.4 Attribut vs Variable locale

★ 1 pt

On crée un objet `p = Point(3, 5)`. Complète les pointillés :

a) Lire l'abscisse depuis l'*extérieur* de la classe : `valeur = p. _____`

b) Lire l'abscisse depuis *une méthode* de la classe : `valeur = _____ .x`

⚠ **Rappel** : Un attribut ne s'appelle **jamais** avec des parenthèses — ce n'est pas une méthode.

Exo 2.1 Méthode de validation – Étudiant



3 pts

La classe `Etudiant` possède deux attributs : `note` (float, 0–20) et `absences` (int).

Écris la méthode `est_admis(self)` qui retourne `True` si la note est **supérieure ou égale à 10** et le nombre d'absences est **strictement inférieur à 5**, `False` sinon.

⚠ **Règles** : Opérateurs : `>=` et `<` · Opérateur logique : `and` en minuscules (jamais `AND` ou `&&`) · Cette méthode **doit** comporter un `return`

TA RÉPONSE

2 À 5 LIGNES

1

2

3

4

5

Exo 2.2 Méthode de catégorie – Produit



3 pts

La classe `Produit` a un attribut `prix` (float). Écris la méthode `categorie(self)` qui retourne :

- `"économique"` si le prix est strictement inférieur à 20
- `"standard"` si le prix est entre 20 et 100 inclus
- `"premium"` sinon

⚠ **Règles** : Utiliser `if / elif / else` — pas des `if` successifs · Chaque branche contient un `return`

TA RÉPONSE

6 À 8 LIGNES

1

2

3

4

5

6

7

8

Exo 2.3 Méthode de modification – Température



3 pts

La classe `Thermostat` a un attribut `temperature` (float). Écris la méthode `chauffer(self, delta)` qui **augmente** la température de `delta` degrés, mais **uniquement** si le résultat ne dépasse pas 30°C.

⚠ **Règles** : Une méthode de *modification* ne renvoie rien — **pas de** `return` · Modifier l'attribut avec `self.temperature = ...`

TA RÉPONSE

4 À 5 LIGNES

1

2

3

4

5

Exo 3.1 **Getter et Setter – Batterie**

★★★

3 pts

La classe `Batterie` possède un attribut `charge` (int, entre 0 et 100). Écris :

- La méthode `get_charge(self)` qui retourne la charge actuelle.
- La méthode `set_charge(self, nouvelle_charge)` qui modifie la charge **seulement si** `nouvelle_charge` est entre 0 et 100 inclus.

⚠ **Règles** : Le getter utilise `return` · Le setter **n'en utilise pas** · Tester avec `0 ≤ nouvelle_charge ≤ 100`

TA RÉPONSE

6 À 8 LIGNES

1

2

3

4

5

6

7

8

Exo 3.2 **Méthode de comparaison – Rectangle**

★★★

3 pts

La classe `Rectangle` possède deux attributs : `largeur` et `hauteur`. Écris :

- La méthode `aire(self)` qui retourne l'aire.
- La méthode `est_plus_grand(self, autre)` qui retourne `True` si l'aire de `self` est **strictement supérieure** à l'aire de `autre`.

⚠ **Règle** : Dans `est_plus_grand`, appelle la méthode `aire()` avec ses parenthèses sur `self` et sur `autre`.

TA RÉPONSE

6 LIGNES

1

2

3

4

5

6

Exo 3.3 **Méthode `__str__` – Affichage d'objet**

★★★

3 pts

La classe `Eleve` a les attributs `nom` et `note`. Écris la méthode spéciale `__str__(self)` qui retourne une chaîne du type `"Alice : 14/20"`.

⚠ **Règle** : `__str__` doit toujours `return` une chaîne — **jamais** utiliser `print` à l'intérieur.

TA RÉPONSE

2 À 3 LIGNES

1

2

3

Exo 4.1 **Cumul d'attributs – Bibliothèque**

★★★★★

3 pts

La classe `Livre` possède les attributs `titre` (str) et `pages` (int). On a une liste `bibliotheque` contenant plusieurs objets `Livre`.

Écris la fonction `total_pages(bibliotheque)` qui retourne le nombre total de pages de tous les livres.

⚠ **Règles** : C'est une *fonction* (hors classe), pas une méthode · Accéder à l'attribut sans parenthèses : `livre.pages` et non `livre.pages()` · Initialiser le compteur à `0` avant la boucle · Le `return` se place **après** la boucle

TA RÉPONSE

5 LIGNES

1

2

3

4

5

Exo 4.2 **Appel de méthode en boucle – Promotion**

★★★★★

3 pts

La classe `Eleve` possède l'attribut `note` et la méthode `est_admis()` (retourne un booléen). Écris la fonction `compter_admis(promo)` qui prend une liste d'objets `Eleve` et retourne le nombre d'élèves admis.

⚠ **Règles** : Appeler une méthode nécessite des parenthèses : `eleve.est_admis()` · Ne pas réécrire la condition — la méthode existe, utilise-la

TA RÉPONSE

6 LIGNES

1

2

3

4

5

6

Exo 4.3 **Recherche du maximum – Tournoi**

★★★★★

3 pts

La classe `Joueur` possède les attributs `nom` (str) et `score` (int). Écris la fonction `meilleur_joueur(joueurs)` qui retourne l'**objet** `Joueur` ayant le score le plus élevé. On suppose que la liste est non vide.

⚠ **Règles** : Initialiser le meilleur candidat avec le premier élément : `meilleur = joueurs[0]` · Retourner l'*objet entier*, pas uniquement son score

TA RÉPONSE

6 LIGNES

1

2

3

4

5

6

On modélise un film avec la classe suivante (partiellement écrite) :

```
class Film:
    def __init__(self, titre, duree, note_presse, note_public):
        self.titre = titre
        self.duree = duree # en minutes
        self.note_presse = note_presse # sur 10
        self.note_public = note_public # sur 10
```

Question 1. (2 pts) Ajoute la méthode `note_globale(self)` qui retourne la moyenne pondérée : **40% presse + 60% public**.

QUESTION 1 – NOTE_GLOBALE

2 À 3 LIGNES

1

2

3

Question 2. (3 pts) Écris la fonction `films_premium(catalogue)` qui prend une liste d'objets `Film` et retourne la **liste** de tous les films dont la note globale est supérieure ou égale à 7.

⚠ **Règles :** Appeler `note_globale()` avec ses parenthèses · Retourner une *liste d'objets Film* · Construire avec `resultat = []` puis `resultat.append(...)`

QUESTION 2 – FILMS_PREMIUM

6 LIGNES

1

2

3

4

5

6

On dispose de la classe `Compte` suivante :

```
class Compte:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde = solde

    def est_debiteur(self):
        return self.solde < 0

    def prelever(self, montant):
        if montant > 0 and self.solde ≥ montant:
            self.solde = self.solde - montant
```

Question 1. (2 pts) Écris la fonction `solde_total(comptes)` qui retourne la somme de tous les soldes (positifs et négatifs).

QUESTION 1 – SOLDE_TOTAL

5 LIGNES

1

2

3

4

Question 2. (3 pts) Écris la fonction `titulaires_debiteurs(comptes)` qui retourne la *liste des noms* (attribut `titulaire`) des comptes débiteurs.

⚠ **Règles :** Utiliser la méthode `est_debiteur()` — ne pas réécrire la condition · Retourner une liste de *chaînes* (noms), pas d'objets

QUESTION 2 — TITULAIRES_DEBITEURS

6 LIGNES

1

2

3

4

5

6

Exo 5.3 Classe Météo — Classe complète + analyse



5 pts

On modélise des relevés météorologiques. Chaque relevé possède : un `jour` (str), une `temperature` (float) et des `precipitations` (float, en mm).

Question 1. (2 pts) Écris la classe `Releve` complète avec son constructeur et la méthode `est_pluvieux(self)` qui retourne `True` si les précipitations sont **strictement supérieures à 5 mm**.

QUESTION 1 — CLASSE RELEVÉ

10 LIGNES

1

2

3

4

5

6

7

8

9

10

Question 2. (1 pt) Écris la fonction `temperature_moyenne(reLeves)` qui calcule la moyenne des températures. Si la liste est vide, retourner `0`.

⚠ **Règle :** Protéger contre la division par zéro avec `if len(reLeves) == 0`

QUESTION 2 — TEMPERATURE_MOYENNE

7 LIGNES

1

2

3

4

5

6

7

Question 3. (2 pts) Écris la fonction `jours_pluvieux(reLeves)` qui retourne la liste des *jours* (attribut `jour`) où il a plu.

⚠ **Règle :** Appeler `est_pluvieux()` avec ses parenthèses

QUESTION 3 – JOURS_PLUVIEUX

6 LIGNES

1

2

3

4

5

6

BARÈME RÉCAPITULATIF

TOTAL : 50 POINTS

Partie	★ (8 pts)	★★ (9 pts)	★★★ (9 pts)	★★★★ (9 pts)	★★★★★ (15 pts)
Note / pts	/ 8	/ 9	/ 9	/ 9	/ 15
Total	Note finale: / 50 → / 20:				